

Experiment ‘p_5_5_repeat’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 0

Soft fails (resolution): 2

Verification attempts: 1

Problem Specification

Problem name: p_5_5_repeat

Natural language statement: null

Method signature: p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

Ensures

- $|repeated_str| == n * |str|$
- forall $i :: 0 \leq i < n \implies repeated_str[i * |str| .. (i + 1) * |str|] == str$

Requires

- $n \geq 0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_5_repeat(str: string, n: int) returns (repeated_str: string)`

The method should respect the following contract:

requires `n >= 0`, ensures `|repeated_str| == n * |str|`, ensures forall `i :: 0 <= i < n ==>`
 `↪ repeated_str[i * |str| .. (i + 1) * |str| ] == str`

Produce and show only the Dafny body of this method, including the curly braces that
 `↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor
 `↪` 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQvXl1fCIXpId7JZmYMPKsVkiRsQ

Model: gpt-5.5-2026-04-23

Created at: 1780673951

Finish reason: stop

Usage: 342 tokens in, and 9093 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
      ↪ (i + 1) * |str| <= |repeated_str|
    invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var prev := repeated_str;
    assert |prev| == k * |str|;
    assert forall i :: 0 <= i < k ==> prev[i * |str| .. (i + 1) * |str|] == str;

    repeated_str := prev + str;
    assert repeated_str == prev + str;
    assert |repeated_str| == |prev| + |str|;
    assert |repeated_str| == (k + 1) * |str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..] == str;
  }
}
```

```

assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  ↪ && (i + 1) * |str| <= |repeated_str| by {
  forall i | 0 <= i < k + 1
    ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |
      ↪ repeated_str|
    {
      assert 0 <= |str|;
      assert 0 <= i;
      assert i <= i + 1;
      assert i + 1 <= k + 1;
      assert i * |str| <= (i + 1) * |str|;
      assert (i + 1) * |str| <= (k + 1) * |str|;
      assert (k + 1) * |str| == |repeated_str|;
    }
  }

forall i | 0 <= i < k + 1
  ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  var a := i * |str|;
  var b := (i + 1) * |str|;
  assert 0 <= a;
  assert a <= b;
  assert b <= |repeated_str|;

  if i < k {
    assert i + 1 <= k;
    assert b <= k * |str|;
    assert b <= |prev|;
    assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
    assert repeated_str[a..b] == prev[a..b];
    assert prev[a..b] == str;
  } else {
    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert repeated_str[a..b] == repeated_str[|prev|..|prev| + |str|];
    assert repeated_str[|prev|..|prev| + |str|] == str;
  }
}

k := k + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_5_5_repeat/program.dfy(11,14): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
```

```
|
11 | invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  ↳ && (i + 1) * |str| <= |repeated_str|
  | ~~~~~
```

```
problem_data/p_5_5_repeat/program.dfy(25,11): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
```

```
|
25 | assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  ↳ && (i + 1) * |str| <= |repeated_str| by {
  | ~~~~~
```

```
problem_data/p_5_5_repeat/program.dfy(26,6): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
```

```
|
26 | forall i | 0 <= i < k + 1
  | ~~~~~
```

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQydL1LgduJUOAY7x9hZkQUtwAwf

Model: gpt-5.5-2026-04-23

Created at: 1780674143

Finish reason: stop

Usage: 1967 tokens in, and 6463 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
```

```

invariant |repeated_str| == k * |str|
invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  var prev := repeated_str;
  assert |prev| == k * |str|;

  forall i | 0 <= i < k
    ensures prev[i * |str| .. (i + 1) * |str|] == str
  {
    var a := i * |str|;
    var b := (i + 1) * |str|;
    assert 0 <= |str|;
    assert 0 <= i;
    assert i + 1 <= k;
    assert 0 <= a;
    assert a <= b;
    assert b <= k * |str|;
    assert b <= |prev|;
    assert prev == repeated_str;
    assert prev[a..b] == repeated_str[a..b];
    assert repeated_str[a..b] == str;
  }

  repeated_str := prev + str;
  assert repeated_str == prev + str;
  assert |repeated_str| == |prev| + |str|;
  assert |repeated_str| == (k + 1) * |str|;
  assert repeated_str[..|prev|] == prev;
  assert repeated_str[|prev|..] == str;

  forall i | 0 <= i < k + 1
    ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var a := i * |str|;
    var b := (i + 1) * |str|;
    assert 0 <= |str|;
    assert 0 <= i;
    assert i + 1 <= k + 1;
    assert 0 <= a;
    assert a <= b;
    assert b <= (k + 1) * |str|;
    assert b <= |repeated_str|;

    if i < k {
      assert i + 1 <= k;
      assert b <= k * |str|;
      assert b <= |prev|;
      assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
      assert repeated_str[a..b] == prev[a..b];
      assert prev[a..b] == str;
    } else {

```

```

    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert b == |repeated_str|;
    assert repeated_str[a..b] == repeated_str[|prev|..|prev| + |str|];
    assert repeated_str[|prev|..|prev| + |str|] == repeated_str[|prev|..];
    assert repeated_str[|prev|..] == str;
  }
}

  k := k + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_5_repeat/program.dfy(17,18): Error: upper bound below lower bound or
↪ above length of sequence

```

```

|
17 | ensures prev[i * |str| .. (i + 1) * |str|] == str
| ^

```

```

problem_data/p_5_5_repeat/program.dfy(26,6): Error: assertion might not hold

```

```

|
26 | assert b <= k * |str|;
| ~~~~~

```

```

problem_data/p_5_5_repeat/program.dfy(41,26): Error: lower bound out of range

```

```

|
41 | ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
| ^

```

Dafny program verifier finished with 1 verified, 3 errors

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR2W39vz7L6uobRh2yNj6cQsHTdq

Model: gpt-5.5-2026-04-23

Created at: 1780674384

Finish reason: stop

Usage: 2763 tokens in, and 6198 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
      ↪ (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|
      ↪ ] == str
  {
    var prev := repeated_str;

    assert forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i
      ↪ + 1) * |str| <= |prev| && prev[i * |str| .. (i + 1) * |str|] == str by {
      forall i | 0 <= i < k
        ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |prev
          ↪ | && prev[i * |str| .. (i + 1) * |str|] == str
      {
        var a := i * |str|;
        var b := (i + 1) * |str|;
        assert prev == repeated_str;
        assert 0 <= a && a <= b && b <= |repeated_str| && repeated_str[a..b] == str;
        assert |prev| == |repeated_str|;
        assert b <= |prev|;
        assert prev[a..b] == repeated_str[a..b];
      }
    }

    repeated_str := prev + str;
    assert repeated_str == prev + str;
    assert |repeated_str| == |prev| + |str|;
    assert |prev| == k * |str|;
    assert |repeated_str| == (k + 1) * |str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..] == str;

    assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
      ↪ && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |
      ↪ str|] == str by {
      forall i | 0 <= i < k + 1
        ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |
          ↪ repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|] == str
      {
        var a := i * |str|;
        var b := (i + 1) * |str|;
        if i < k {
          assert 0 <= a && a <= b && b <= |prev| && prev[a..b] == str;
          assert |str| >= 0;
          assert |prev| <= |repeated_str|;
        }
      }
    }
  }
}
```

```

    assert b <= |repeated_str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
    assert (repeated_str[..|prev|])[a..b] == prev[a..b];
    assert repeated_str[a..b] == str;
  } else {
    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert b == |repeated_str|;
    assert 0 <= a;
    assert a <= b;
    assert repeated_str[a..b] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[|prev|..|repeated_str|] == repeated_str[|prev|..];
    assert repeated_str[a..b] == str;
  }
}
}

k := k + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_5_5_repeat/program.dfy(11,14): Warning: Could not find a trigger for this
  → quantifier. Without a trigger, the quantifier may cause brittle verification. To
  → silence this warning, add an explicit trigger using the {:trigger} attribute. For
  → more information, see the section on quantifier instantiation rules in the
  → reference manual.
|
11 | invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  → && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|]
  → == str
| ~~~~~
problem_data/p_5_5_repeat/program.dfy(15,11): Warning: Could not find a trigger for this
  → quantifier. Without a trigger, the quantifier may cause brittle verification. To
  → silence this warning, add an explicit trigger using the {:trigger} attribute. For
  → more information, see the section on quantifier instantiation rules in the
  → reference manual.
|
15 | assert forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
  → (i + 1) * |str| <= |prev| && prev[i * |str| .. (i + 1) * |str|] == str by {
| ~~~~~

```



```

problem_data/p_5_5_repeat/program.dfy(16,6): Warning: Could not find a trigger for this
    ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    ↪ more information, see the section on quantifier instantiation rules in the
    ↪ reference manual.
|
16 | forall i | 0 <= i < k
|   ~~~~~

problem_data/p_5_5_repeat/program.dfy(37,11): Warning: Could not find a trigger for this
    ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    ↪ more information, see the section on quantifier instantiation rules in the
    ↪ reference manual.
|
37 | assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
    ↪ && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|
    ↪ |] == str by {
|   ~~~~~

problem_data/p_5_5_repeat/program.dfy(38,6): Warning: Could not find a trigger for this
    ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    ↪ more information, see the section on quantifier instantiation rules in the
    ↪ reference manual.
|
38 | forall i | 0 <= i < k + 1
|   ~~~~~

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR4trcvwCFWYRS411bpZIa7bXPWB

Model: gpt-5.5-2026-04-23

Created at: 1780674531

Finish reason: length

Usage: 5074 tokens in, and 3174 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
```

```

    requires n >= 0
    ensures |repeated_str| == n * |str|
    ensures forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] ==
        ↪ str
{
    repeated_str := "";
    var k := 0;
    while k < n
        invariant 0 <= k <= n
        invariant |repeated_str| == k * |str|
        invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
            ↪ (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|
            ↪ ] == str
        {
            var prev := repeated_str;

            assert forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i
                ↪ + 1) * |str| <= |prev| && prev[i * |str| .. (i + 1) * |str|] == str by {
                forall i | 0 <= i < k
                    ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |prev|
                    ↪ | && prev[i * |str| .. (i + 1) * |str|] == str
                {
                    var a := i * |str|;
                    var b := (i + 1) * |str|;
                    assert prev == repeated_str;
                    assert 0 <= a && a <= b && b <= |repeated_str| && repeated_str[a..b] == str;
                    assert |prev| == |repeated_str|;
                    assert b <= |prev|;
                    assert prev[a..b] == repeated_str[a..b];
                }
            }

            repeated_str := prev + str;
            assert repeated_str == prev + str;
            assert |repeated_str| == |prev| + |str|;
            assert |prev| == k * |str|;
            assert |repeated_str| == (k + 1) * |str|;
            assert repeated_str[..|prev|] == prev;
            assert repeated_str[|prev|..] == str;

            assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
                ↪ && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |
                ↪ str|] == str by {
                forall i | 0 <= i < k + 1
                    ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |
                    ↪ repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|] == str
                {
                    var a := i * |str|;
                    var b := (i + 1) * |str|;
                    if i < k {
                        assert 0 <= a && a <= b && b <= |prev| && prev[a..b] == str;

```

```

    assert |str| >= 0;
    assert |prev| <= |repeated_str|;
    assert b <= |repeated_str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
    assert (repeated_str[..|prev|])[a..b] == prev[a..b];
    assert repeated_str[a..b] == str;
  } else {
    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert b == |repeated_str|;
    assert 0 <= a;
    assert a <= b;
    assert repeated_str[a..b] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[|prev|..|repeated_str|] == repeated_str[|prev|..];
    assert repeated_str[a..b] == str;
  }
}
}

k := k + 1;
}
assert k == n;
}

```

Total Token Usage

Input tokens: 10146

Output tokens: 24928

Reasoning tokens: 22074

Sum of 'total tokens': 35074

Experiment Timings

Overall Experiment started at 1780673950879, ended at 1780674624892, lasting 674013ms (674.01 seconds)

Iteration #4 started at 1780674530861, ended at 1780674624876, lasting 94015ms (94.02 seconds)

Iteration #1 started at 1780673950881, ended at 1780674142558, lasting 191677ms (191.68 seconds)

Iteration #2 started at 1780674142561, ended at 1780674379681, lasting 237120ms (237.12 seconds)

Iteration #3 started at 1780674379691, ended at 1780674530861, lasting 151170ms (151.17 seconds)